AJAX Requests

Contents

- 11.1. AJAX
- 11.2. Fetching Data
- 11.3. Asynchronous Programming
- 11.4. Resources
- 11.5. Original Info 340 Acknowledgements

JavaScript allows you to dynamically define the content of a web page, generating the DOM at runtime rather than in `.html` source files. One of the primary reasons we would want to dynamically produce a DOM is if the web page's content is based on some **data** that may change over time: for example, the kind of data that is available through a **Web API**. By using JavaScript to render the DOM, you can quickly produce large amounts of HTML needed to display large data sets, sure that you have up-to-date data each time the page loads, and even *automatically refresh the page content* without requiring the user to reload!

This chapter describes how to use JavaScript to dynamically send HTTP Requests to download data (without reloading the page!), as well as how to perform the **asynchronous programming** needed when working with web requests and other time-consuming operations.

11.1. AJAX

As discussed in the Overview, you download data from the Internet by sending an **HTTP Request** and then processing the **response**. In everyday usage, HTTP Requests are usually sent by the *browser* when the user enters a URL or clicks on a link. By default, if you wanted to download new data, you'd need to have the browser send a new request, loading a new page (or reloading the current page) to show that result.

To make modern dynamic web pages that display new data without needing to refresh the browser, we use a technique to send HTTP Requests *from JavaScript code* rather than from the browser. This allows us to "bypass" the browser and get new data (and change the webpage) without reloading it! This technique is referred to as **AJAX** (**A**synchronous **J**avaScript **A**nd **X**ML) — we write code that sends an "request with AJAX" or an "AJAX request".

Fun Fact

The technology used to send AJAX requests was initially developed by Microsoft in the late 90s to support their fledgling web version of the Outlook email/calendar app. The JavaScript functions used to send these requests were included in Internet Explorer as a *non-standard* feature — an example of a browser adding new functionality that it thinks will be useful but doesn't work on other platforms. However, AJAX quickly gained popularity (particularly when Google showed off what you could do with it via Gmail and Google Maps), and has since become a standard that is now supported by all browsers. This is how standards come into existence!

11.1.1. XML and JSON

AJAX is called "AJA**X**" because it was initially designed to request data in XML format. **XML** (E**X**tensible **M**arkup **L**anguage) is a markup language (like HTML) used to encode meaning in content in a human *and* computer-readable format. The syntax for XML is *exactly* the same as HTML: in fact, HTML can be seen as a "subset" of the language. You can think of XML as "HTML, but you get to make up your element names!"

```
<!-- Some XML encoding information about a person -->
<person>
   <firstName>Alice</firstName>
   <lastName>Smith</lastName>
   <favorites>
      <music>jazz</music>
      <food>pizza</food>
      <numbers>
          <item>12</item>
          <item>42</item>
      </numbers>
   </favorites>
</person>
```

The XML language does not define any particular tags the way HTML does; instead, it is up to individual applications to determine what tags it will recognize and interpret (and what tags it would see as gibberish) &mdash, which is referred to as a XML Schema.

When AJAX was first developed, XML was the most common way of encoding generic data for transmission. And because XML is a tree of elements just like the DOM, similar methods could be used to navigate and extract information from the tree. However, XML is a very *verbose* language: it requires a lot of characters to encode information (meaning that larger amounts of data are transferred), and traversing an element tree requires a lot of code. As such, JavaScript developers (led by Douglas Crockford) developed an alternative language called **JSON** (**J**ava**S**cript **O**bject **N**otation) that is more compact than XML and can be *directly* parsed into JavaScript objects and arrays:

```
{
  "firstName": "Alice",
  "lastName": "Smith",
  "favorites": {
    "music": "jazz",
    "food": "pizza",
    "numbers": [12, 42]
  }
}
```

JSON format uses a syntax that is almost identical to that for defining Object literals in JavaScript, with a few key differences:

- JSON defines an Object `{}` at the "top-level". However, an Object can also be an array (`[]`).
- JSON object **keys** (which must be strings) *must* be written in double-quotes.

- JSON **values** can only be strings, numbers, booleans (`true` or `false`), arrays (`[]`), or other objects. You cannot include a function in JSON.
- JSON objects and arrays can't have trailing commas or other extraneous symbols — no comments!

The JavaScript language provides a global object `JSON` (like the global `Math` object) that can be used to convert from encoded *strings* of JSON content (e.g., the above code block as a single string variable `'{"firstName":"Alice"}'`) to JavaScript objects, and vice versa:

```
//convert from Object to encoded String
let personObj = {firstName:"Alice", lastName:"Smith", id:12} //JavaScript
let personString = JSON.stringify(personObj); //turn object into JSON stri
console.log(personString); //=> '{"firstName":"Alice","lastName":"Smith","
console.log(typeof personString); //=> 'string'

//convert from encoded String to Object
let favoritesString = '{"music":"jazz", "numbers":[12,42]}'; //a string, n
let favoritesObj = JSON.parse(favoritesString); //turn JSON string into ob
console.log(favoritesObj); //=> { music: 'jazz', numbers: [ 12, 42 ] }
console.log(typeof favoritesObj); //=> 'object'
```

> **Note**
>
> If your JSON string is not properly formatted (e.g., you're missing a quote), the `JSON.parse()` function will throw a `SyntaxError`. The exact error in the JSON string can be hard to find; online tools can help show the problem.

JSON has replaced XML as the encoding of choice for working with AJAX requests — however, the technique is still called "AJAX" ("AJA*J*" isn't as easy to say!)

11.2. Fetching Data

AJAX support is built into browsers through the included `XMLHttpRequest` global variable (the "xml http thing"). This object provides functions that allow you to send an HTTP request to the server, but the object's API is **complex to use**:

Example XMLHTTPRequest

```
//create a new XMLHttpRequest object
let request = new XMLHttpRequest();

//configure it to do an HTTP GET request for some URL
request.open('GET', 'https://domain.com/data', true);

//add a listener for the "load" event (when the data has been down
request.addEventListener('load', function() {
    if (request.status >= 200 && request.status < 400) { //check r
        let data = JSON.parse(request.responseText);
        console.log(data); //do something with the data
    }
});

//listen for "error" events if there was a network error
request.addEventListener('error', function() {
    //handle error...
})

//finally, send the request to the server!
request.send();
```

Instead of needing to understand all that code, developers tended to use functions from external libraries such as jQuery's `$.getJSON()` or `$.ajax()`:

```
$.getJSON('https://domain.com/data', function(data) {
    //`data` is the already-parsed JSON data
    console.log(data); //do something with the data
});
```

But this requires including the jQuery library in your page, and since the need for jQuery is rapidly going away, other options are now *built in* to modern browsers. Now, use the `fetch()` API to easily send AJAX requests for data! The `fetch()` function makes it easy to send a request: simply call it and pass in the URL of the data you wish to download:

```
fetch('https://domain.com/data');
```

Warning

In some browsers, you can not send an AJAX request when the web page is loaded via the `file://` protocol (e.g., by double-clicking on the `.html` file). This is a security feature to keep you from accidentally running an HTML file containing malicious JavaScript that will download a virus onto your computer.

You will need to run a local web server such as Python's http.server module: `python -m http.server`

Tip

Use a **relative path** when specifying what file or API you wish to fetch. Moreover, paths are always relative to the `.html` file the user is currently viewing, not to the `.js` file containing the `fetch()` call. So be careful if your JavaScript is in a different folder; the path must be relative to the HTML (which should be in the root of your project).

11.3. Asynchronous Programming

However, the `fetch()` function does **NOT** directly return the data you want to download! Downloading data off the internet can take a long time: the network connection may be slow and the amount of data to download may be quite large (metadata for the latest 100 tweets from Twitter involves almost 500k of JSON content). Because fetching data may take time, AJAX requests are made **asynchronously** (that's the "A" in AJAX) — the download will occur *at the same time* that the rest of the code is being executed. Thus, the download and the remaining script will *not* be synchronized; they will be "asynchronous".

```
console.log('About to send request'); //statement 1

//send request for data to the url
fetch(url);  //statement 2

console.log('Sent request'); //statement 3

//The data is actually received sometime later,
//when the JS interpreter is down here!
```

In the above example, the JS interpreter will execute statement 1, then statement 2 (the `fetch()` call). It precedes *immediately* to statement 3 (the second `console.log`), without waiting for the request to finish! The download will continue to occur in the background, and will finish at some point later in the program — though we don't know exactly when.

> **Tip**
>
> Think of `fetch()` as a function that will just "*start* to download data", not one that actually downloads data!

Because `fetch()` is an **asynchronous function** (the code runs asynchronously), it returns what is called a **Promise**. A Promise is an object that holds a value which may not be available yet — you can think of a Promise as like a placeholder where the result of the asynchronous function call will eventually be stored (it is a "promise" to eventually have some data, though that promise may be kept or broken!).

Promises are the modern way of handling asynchronous functions. They have three possible states:

- *pending* (the data is downloading)
- *fulfilled* (the data has finished downloading)
- *rejected* (the data failed to download and the promise was "broken").

We specify *callback functions* (similar to event listeners) that occur when a pending Promise is either successfully fulfilled or has been rejected. The "on success" callback function is specified by calling the `.then()` function on the Promise object, and passing the "on success callback" as a parameter:

```
function onSuccessCallback(response) { //what to do when we get the respon
    console.log(response);
}

//When fulfilled, execute the callback function (which will be passed the
let promise = fetch(url);
promise.then(onSuccessCallback);

//It is much more common to use anonymous variables/callbacks:
fetch(url).then(function(response) {
    console.log(response);
});
```

The “on success” callback will be passed a single parameter: the **data value** that the Promise was made for (e.g., the data that will eventually be downloaded from `fetch()`). So when the callback is executed, you will have access to the data! For example, when the `fetch()` Promise is fulfilled, it will pass an object representing the *response* to the HTTP Request:

```
let promise = fetch(url);
promise.then(function(response){
    console.log( response.url ); //a string of where the request was sent
    console.log( response.status ); //the HTTP status code (e.g., 200, 404
});
```

This response object has a `body` property that represents the “body” (data content) of the HTTP response. However, that body is a stream of data. You’ll need to look at the `Content-Type` header to determine how to parse that data:

```
fetch('https://api.example.com/data')
  .then(response => {
    // Access the Content-Type header
    const contentType = response.headers.get('Content-Type');
    console.log(`Content-Type: ${contentType}`);

    // Parse the body based on the Content-Type
    if (contentType && contentType.includes('application/json')) {
      return response.json(); // Parse as JSON
    } else if (contentType && contentType.includes('text/plain')) {
      return response.text(); // Parse as text
    } else {
      throw new Error('Unsupported Content-Type');
    }
  })
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

As you can see above, you can convert the body in a `json` object by calling the `.json()` method on the *response* object.

An equivalent `.text()` encodes a response body into plain text.

11.3.1. Chaining Promises

But there's a catch: the "encoding" process performed by the `.json()` might take some time (particularly for a large amount of data). So instead of blocking (pausing) the rest of your program while that encoding occurs, the `.json()` method returns *another Promise* as a placeholder for when the encoded body is available! So you will then need to specify a `.then()` callback for *that* Promise as well.

However, a Promise's `.then()` function has a neat property that makes this easy to do. Calling the `.then()` function on a Promise returns a new Promise as a placeholder for any data produced by the `.then()` function. This promised data will be whatever value is *returned* by the "on success" callback function. This allows you to "chain" `.then()` calls together, each of which performs some kind of transformation/operation on the data:

```
function makeQuestion(dataString) { //a function to make a string a questi
    return dataString + '???';
}

//image a hypothetical asynchronous function `getAsyncString`
//it returns a Promise (placeholder) for a string load from a given source
let originalPromise = getAsyncString(myDataSource);

//when the original promise is fulfilled, call `makeQuestion` on it
//`questionPromise` will be a placeholder for that transformed data
let questionPromise = originalPromise.then(makeQuestion);

//when the `questionPromise` is fulfilled, call an anonymous callback on i
//the callback will be passed the transformed ("question") data
questionPromise.then(function(data){
    console.log(data); //data will be a question!
})
```

More commonly, we use *anonymous variables* for subsequent promises, allowing you to chain them together in a way that almost reads like English!

```
getAsyncString()
    .then(makeQuestion)
    .then(function(data){
        console.log(data);
    });
```

`.then()` also has a unique feature where if the "on success" callback function returns a *Promise* (rather than another kind of value), then the "outer" promise will take on the state of that new returned Promise. This means that you can *return* a Promise from inside a `.then()` callback, and that Promise will be the subject of the subsequent `.then()` call:

```
let outerPromise = getAsyncString(myFirstSource).then(function(firstData){
    //do something with `firstData`

    let newPromise = getAsyncString(mySecondSource); //a second async call
    return newPromise; //return the promise.
}); //`outerPromise` now takes on the state and data of `newPromise`

outerPromise.then(function(secondData){
    //do something with `secondData`, the data downloaded from `mySecondSo
});
```

Going back to `fetch()` to bring it all together: since the `.json()` encoding function returns a Promise, you can simply *return* that Promise from the `.then()` callback to make it available to subsequent `.then()` calls!

```
fetch(url)  //start the download
    .then(function(response) {  //when done downloading
        let dataPromise = response.json();  //start encoding into an objec
        return dataPromise;  //hand this Promise up
    })
    .then(function(data) {  //when done encoding
        //do something with the data!!
        console.log(data); //will now be encoded as a JavaScript object!
    });
```

This code example will allow you to download data and encode it into a plain old JavaScript object that you can work with.

11.3.2. Handling Errors

When downloading data from the internet, it is always possible that the HTTP request may fail. The request may be sent to the wrong URL, the client computer may be having connection problems, or the receiving server may be having problems.

In order to deal with inevitable errors, Promises provide a `.catch()` method that is used to specify a callback that should occur if the Promise is *rejected* (an error occurs). This callback function will be passed an Error object that contains details about the error.

```
fetch(url)
    .then(function(response) {  //when done downloading
        return response.json();  //second promise is anonymous
    })
    .then(function(data) {  //when done encoding
        //do something with the data!!
        console.log(data); //will now be encoded as a JavaScript object!
    })
    .catch(function(err) {
        //do something with the error
        console.error(err);  //e.g., show in the console
    });
```

- Importantly, the `.catch()` method will "catch" errors from *all previous* Promises in a `.then()` chain! This means that the above `.catch()` will show both errors in the downloading (`.fetch()`), and errors in the body encoding (`.json()`).
- You will almost always want to show the error to the user in some way, such as by creating an alert element in the DOM.
- The `.catch()` function *also* returns a Promise, so you can continue to chain `.then()` calls after it. These later callbacks will only be executed if no previous Promise has been rejected (that is, there haven't been any errors yet).

Important: a Promise will only be rejected if there is an actual "Error" in sending the request. If the server replies with a 401 error (e.g., you didn't have permission to access the resource) or just the message "invalid API key", that won't be handled by `.catch()`. From JavaScript's perspective, the request went through perfectly — it's not fetch's fault that the data you asked for wasn't what you actually wanted!

- You can use the `response.status` and `response.ok` properties to check the status of the HTTP response.

As such, you will want to make sure to handle things like bad responses or unexpected response bodies, both in testing your application (to make sure the request is sent to the correct URL) and when handling any user input.

11.3.3. Other Data Formats

The above usage of `fetch()` works well for data formatted in JSON (which is the most common format for web-accessible data). However, you may wish to dynamically load data that is presented in a different format, such as plain text or as comma-separated values (CSV). There are a few ways to support this:

For downloading **plain text** formatted data, you can use the `fetch()` method as above, but instead of calling `.json()` on the response to encode it as a JavaScript object, you can call the `.text()` method to encode it as a basic string:

```
fetch(url)  //start the download
    .then(function(response) {  //when done downloading
        let dataPromise = response.text();  //start encoding into a String
        return dataPromise;  //hand this Promise up
    })
    .then(function(text) {  //when done encoding
        //do something with the text data!!
        console.log(text); //will now be encoded as a JavaScript string!
    });
```

Encoding as text will support *any* plaintext formated data — whether from a `.txt` file, a `.csv` file, or even a `.json` or `.js` file! In fact, if you encode JSON data as plain text using the `.text()` method, you could then explicitly parse that into a JavaScript Object by using the built-in `JSON.parse()` function described above!

While the `.text()` method will you encode data into a JavaScript String, that String will often have a particular format that needs to be **parsed** (interpreted) in order to make the data useful. For example, a String in CSV format is not very useful on its own; you would need to parse it into an array of objects (where each object represents a row) to analyze the data.

It is possible (but complex) to do this parsing using built-in String functions (assuming your CSV data matches official standards), but in general it's easier and more effective to use an external library to do this parsing. One of the best libraries for doing this work

is `d3.dsv()`, a component of the d3 visualization library. `d3.fetch()` provides convenience wrapper functions for `fetch()` that also perform effective data parsing.

In order to use `d3.fetch()`, you will need to load it as an external library:

```
<script src="https://d3js.org/d3-dsv.v1.min.js"></script>
<script src="https://d3js.org/d3-fetch.v1.min.js"></script>
```

(More details coming soon...)

11.4. Resources

- An Introduction to AJAX for Front-End Developers (tuts+)
- An Introduction to `fetch()` (Google)
- JavaScript Promises: an Introduction (Google)

11.5. Original Info 340 Acknowledgements

This chapter was originally authored by Joel Ross and Mike Freeman. Some content was originally adapted from tutorials by David Stearns. Some structure and explanations inspired by Learning Web Design by Jennifer Robbins. Minor wordsmithing to assume students have a Python background.